

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: Task 6

Student Name: R Varshini

Reg. No.: A24126552115

Date: 21-09-2026

1. TITLE

File Management in Node.js Using the File System (fs) Module

2. AIM

To create a Node.js application that accepts a filename and content from the user and, using the File System (fs) module, creates and writes the file, reads its contents, appends additional content to it, and displays the final contents of the file.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how to use the built-in **fs** module in Node.js.
- To accept input from the user in the terminal using the **readline** module.
- To create a file and write content into it.
- To read the contents of an existing file.
- To append new content to a file without erasing the existing content.
- To display the final contents of the file and handle errors gracefully.

4. THEORY

Node.js is a JavaScript runtime that executes JavaScript outside the browser. It ships with built-in modules, so no extra installation is needed to work with files. The **fs** (File System) module provides functions to create, read, update and delete files.

- **require()** loads a module into the program (e.g. `require('fs')`).
- **fs.writeFileSync(path, data)** creates the file if it does not exist and replaces its content if it does.
- **fs.readFileSync(path, 'utf8')** reads the whole file; the 'utf8' encoding returns the content as a string instead of a Buffer.
- **fs.appendFileSync(path, data)** adds data at the end of the file and keeps the existing content.
- **Synchronous methods** (names ending in Sync) block the program until the operation finishes, so the steps run in the exact order written. The asynchronous versions (`fs.writeFile`, `fs.readFile`, `fs.appendFile`) use callbacks or promises.
- **readline** reads a line of text typed by the user from standard input (`process.stdin`).
- **Promise / async-await** is used to wait for the user's answer before the next statement executes.
- **try...catch** handles run-time errors such as an invalid path or missing permission.

Flow: User input → `writeFileSync` (create/write) → `readFileSync` (initial contents) → `appendFileSync` (append) → `readFileSync` (final contents)

5. ALGORITHM / PROCEDURE

1. Start. Import the **fs** and **readline** modules.
2. Create a readline interface for reading input from the terminal.
3. Prompt the user for the filename. If it is empty, display an error and stop.
4. Prompt the user for the initial content of the file.
5. Create the file and write the initial content using **fs.writeFileSync()**; display a success message.
6. Read the file using **fs.readFileSync()** and display it as the initial file contents.
7. Prompt the user for the content to append.
8. Append the content using **fs.appendFileSync()**; display a success message.
9. Read the file again and display it as the final file contents.
10. Close the readline interface and stop.
11. Run the program in the terminal with `node pbm1.js` and verify the actual output against the expected output for each test case.

6. PROGRAM

pbm1.js

```
// Node.js File Management using the File System (fs) module
const fs = require('fs');
const readline = require('readline');

// Create an interface to read input from the user
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Helper function: ask a question and return the answer as a Promise
function ask(question) {
  return new Promise((resolve) => rl.question(question, resolve));
}

async function main() {
  try {
    // 1. Accept filename and initial content from the user
    const filename = (await ask('Enter the filename:')).trim();
    if (filename === '') {
      console.error('Error: filename cannot be empty.');
```

```

    console.log(finalData);
  } catch (err) {
    console.error('File operation failed:', err.message);
  } finally {
    rl.close();
  }
}

main();

```

7. CODE EXPLANATION

Code	Explanation
<code>require('fs')</code>	Loads the built-in File System module used for all file operations.
<code>require('readline')</code>	Loads the module used to read text typed by the user in the terminal.
<code>readline.createInterface({...})</code>	Connects the program to standard input and standard output.
<code>ask(question)</code>	Helper function that shows a prompt and returns the user's answer as a Promise.
<code>await ask(...)</code>	Pauses the program until the user types a value and presses Enter.
<code>fs.writeFileSync(filename, content)</code>	Creates the file (or overwrites an existing one) and writes the initial content.
<code>fs.readFileSync(filename, 'utf8')</code>	Reads the complete file and returns its contents as a string.
<code>fs.appendFileSync(filename, extra)</code>	Adds the new text at the end of the file without deleting existing content.
<code>console.log(...)</code>	Displays messages and file contents in the terminal.
<code>try ... catch ... finally</code>	Catches file errors and prints a message; finally always closes the interface.
<code>rl.close()</code>	Closes the input interface so that the program can exit.

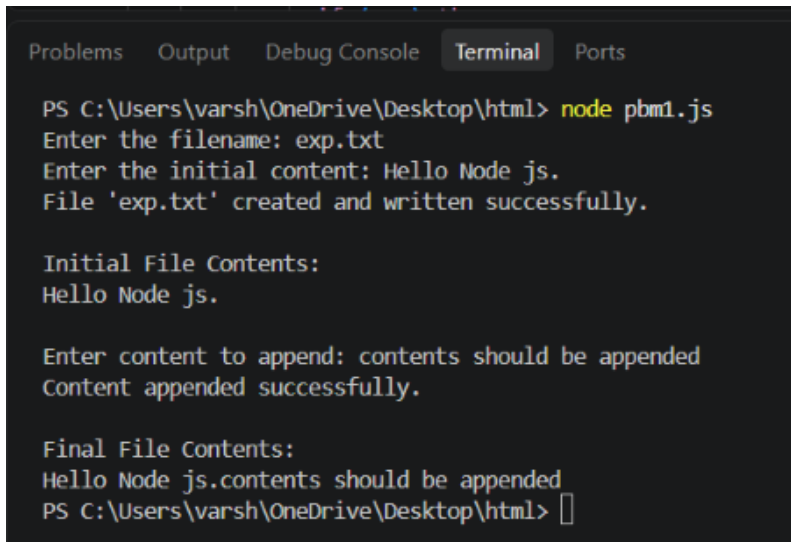
8. TEST CASES AND EXECUTION DATA

Each test case was executed from the terminal using `node pbm1.js` by entering the values below. The final file contents were recorded exactly as displayed.

Test Case	Filename	Initial Content	Append Content	Expected Final Contents	Actual Final Contents	Status
TC-01	exp.txt	Hello Node js.	contents should be appended	Hello Node js.contents should be appended	Hello Node js.contents should be appended	Pass
TC-02	notes.txt	Node.js File System	module is used for file handling.	Node.js File System module is used for file handling.	Node.js File System module is used for file handling.	Pass
TC-03	empty_append.txt	Only initial content.	(nothing - just Enter)	Only initial content.	Only initial content.	Pass
TC-04	(empty)	-	-	Error: filename cannot be empty.	Error: filename cannot be empty.	Pass
TC-05	data.txt (run twice)	Run 1: First run content. Run 2: Second run content.	Run 1: (setup) Run 2: Appended.	Second run replaces the old file: Second run content. Appended.	Second run content. Appended.	Pass

9. OUTPUT

Output 1: TC-01 – exp.txt (run in the VS Code terminal)



```
Problems Output Debug Console Terminal Ports

PS C:\Users\varsh\OneDrive\Desktop\html> node pbm1.js
Enter the filename: exp.txt
Enter the initial content: Hello Node js.
File 'exp.txt' created and written successfully.

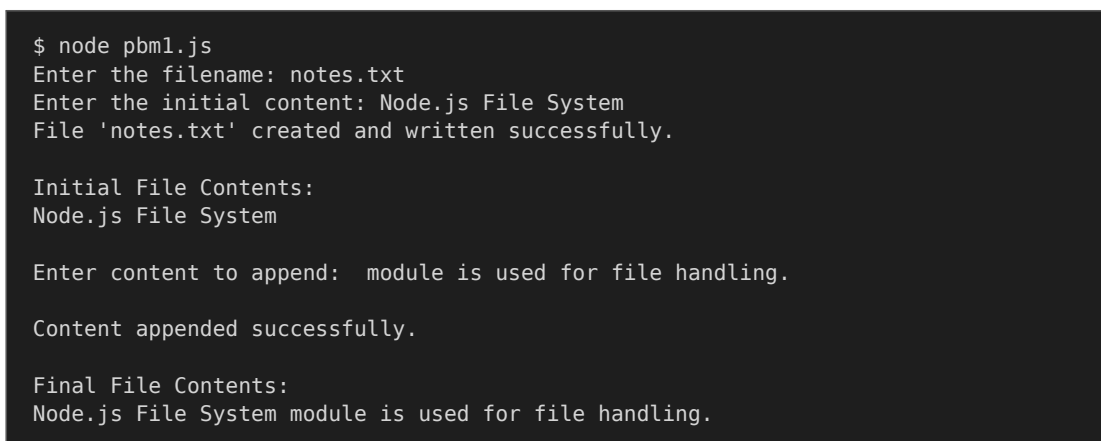
Initial File Contents:
Hello Node js.

Enter content to append: contents should be appended
Content appended successfully.

Final File Contents:
Hello Node js.contents should be appended
PS C:\Users\varsh\OneDrive\Desktop\html> 
```

The file was created, read, appended and read again; the final contents are **Hello Node js.contents should be appended**.

Output 2: TC-02 – notes.txt



```
$ node pbm1.js
Enter the filename: notes.txt
Enter the initial content: Node.js File System
File 'notes.txt' created and written successfully.

Initial File Contents:
Node.js File System

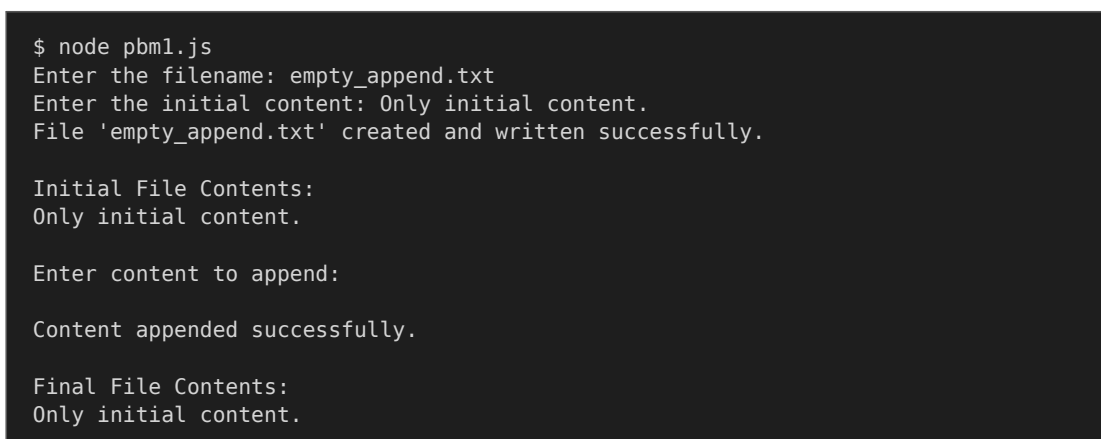
Enter content to append: module is used for file handling.

Content appended successfully.

Final File Contents:
Node.js File System module is used for file handling.
```

The appended text was added after the existing content.

Output 3: TC-03 – nothing appended



```
$ node pbm1.js
Enter the filename: empty_append.txt
Enter the initial content: Only initial content.
File 'empty_append.txt' created and written successfully.

Initial File Contents:
Only initial content.

Enter content to append:

Content appended successfully.

Final File Contents:
Only initial content.
```

Appending an empty string leaves the file unchanged.

Output 4: TC-04 – empty filename

```
$ node pbml.js
Enter the filename:
Error: filename cannot be empty.
```

The program shows an error message and stops without creating any file.

Output 5: TC-05 – data.txt run twice (existing file is overwritten)

```
$ node pbml.js
Enter the filename: data.txt
Enter the initial content: First run content.
File 'data.txt' created and written successfully.

Initial File Contents:
First run content.

Enter content to append: (setup)

Content appended successfully.

Final File Contents:
First run content. (setup)

$ node pbml.js
Enter the filename: data.txt
Enter the initial content: Second run content.
File 'data.txt' created and written successfully.

Initial File Contents:
Second run content.

Enter content to append: Appended.

Content appended successfully.

Final File Contents:
Second run content. Appended.
```

On the second run the earlier content of data.txt is replaced, because writeFileSync overwrites an existing file.

10. RESULT

Thus, a Node.js application was successfully developed that accepts a filename and content from the user and uses the File System module to create and write a file, read its contents, append additional content and display the final contents. The application was verified using multiple test cases.